

Implementacja i Optymalizacja Testu Pierwszości AKS w Języku C

Autor: Marcel Słotwiński

Spis Treści

1. Wstęp i Podstawy Teoretyczne	2
2. Matematyczny Schemat Działania Algorytmu	2
3. Modyfikacje Architektoniczne i Optymalizacje Sprzętowe	3
3.1. Niejawne Sprawdzanie Największego Wspólnego Dzielnika	3
3.2. Pominiecie Redukcji Modulo w Pętli Mnożenia Wielomianów	3
3.3. Całkowitoliczbowa Metoda Babilońska	3
4. Analiza Fragmentów Kodu Źródłowego	3
5. Wydajność Obliczeniowa i Ograniczenia	4
6. Wpływ Hipotezy Agrawala na Efektywność Algorytmu	5
7. Analiza Pomiarów Empirycznych	5
8. Literatura	7

1. Wstęp i Podstawy Teoretyczne

Przez dziesięciolecia efektywne określanie, czy dana liczba jest pierwsza, czy złożona (w czasie wielomianowym) bez opierania się na nieudowodnionych założeniach matematycznych, było jednym z głównych nierozwiązanych problemów informatyki teoretycznej. Krajobraz ten zmienił się diametralnie w 2002 roku, kiedy to Manindra Agrawal, Neeraj Kayal i Nitin Saxena opublikowali swoją przełomową pracę „PRIMES is in P”. Przedstawili w niej pierwszy bezwarunkowy, deterministyczny algorytm czasu wielomianowego do testowania pierwszości liczb.

Algorytm opiera się na bezpośrednim uogólnieniu Małego Twierdzenia Fermata do pierścieni wielomianów nad ciałami skończonymi. Zależy on od matematycznej tożsamości, która stwierdza, że dla dowolnej liczby całkowitej a względnie pierwszej z n , liczba n jest pierwsza wtedy i tylko wtedy, gdy spełniona jest poniższa kongruencja:

$$(X + a)^n \equiv X^n + a \pmod{n}$$

Ponieważ bezpośrednie wyliczenie lewej strony tego równania wymaga obliczenia n współczynników wielomianu (co zajmuje czas $O(n)$ i jest niewykonalne dla liczb stosowanych w kryptografii), autorzy zredukowali to zadanie. Zaproponowali oni obliczanie obu stron kongruencji modulo wielomian postaci $X^r - 1$, gdzie r jest odpowiednio dobraną, niewielką liczbą całkowitą. Rdzeń zaimplementowanego przeze mnie algorytmu testuje zatem następującą zależność:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1, n}$$

Sprawdzając ten warunek dla określonego przedziału wartości parametru a , aż do wyliczonego analitycznie limitu, algorytm zapewnia matematycznie absolutną gwarancję pierwszości badanego wejścia. Głównym celem mojego projektu było przeniesienie tej czystej teorii matematycznej na zoptymalizowany, bezpieczny i wydajny kod w języku C, korzystający z zewnętrznej biblioteki GMP (GNU Multiple Precision Arithmetic Library) do obsługi wielkich liczb całkowitych.

2. Matematyczny Schemat Działania Algorytmu

Poniżej przedstawiam formalny, sformalizowany opis matematyczny poszczególnych kroków algorytmu AKS w jego klasycznej interpretacji, zgodnie z oryginalną publikacją naukową.

Algorytm AKS (Agrawal-Kayal-Saxena) krok po kroku:

Wejście: Liczba całkowita $n > 1$.

1. **Test doskonałej potęgi:** Jeśli $n = a^b$ dla $a \in \mathbb{N}$ oraz $b > 1$, zwróć wynik **ZŁOŻONA**.
2. **Szukanie parametru r :** Znajdź najmniejszą liczbę r taką, że rząd liczby n modulo r (oznaczany jako $o_r(n)$) spełnia nierówność $o_r(n) > \log^2 n$.
3. **Sprawdzenie dzielników:** Jeśli dla jakiegokolwiek liczby $a \leq r$ zachodzi warunek $1 < \gcd(a, n) < n$, zwróć wynik **ZŁOŻONA**.
4. **Przypadek graniczny:** Jeśli $n \leq r$, zwróć wynik **PIERWSZA**.
5. **Główna pętla kongruencji:** Dla każdej liczby całkowitej a od 1 do $\lfloor \sqrt{\varphi(r)} \log_2 n \rfloor$ wykonaj test:
 - Jeśli $(X + a)^n \not\equiv X^n + a \pmod{X^r - 1, n}$, zwróć wynik **ZŁOŻONA**.
6. **Wynik ostateczny:** Jeśli liczba n przetrwa wszystkie powyższe weryfikacje bez wcześniejszego przerwania programu, zwróć wynik **PIERWSZA**.

3. Modyfikacje Architektoniczne i Optimalizacje Sprzętowe

Tłumaczenie akademickiej matematyki dyskretnej n kod w języku C wymagało ode mnie podjęcia kilku kluczowych decyzji inżynierskich. Zaimplementowałem szereg krytycznych optymalizacji niskopoziomowych, które zwiększają kulturę pracy programu na nowoczesnych architekturach CPU.

3.1. Niejawne Sprawdzanie Największego Wspólnego Dzielnika

Krok 3 w oryginalnej pracy naukowej wymaga jawnego sprawdzenia, czy zachodzi warunek $1 < \gcd(a, n) < n$ dla wszystkich wartości $a \leq r$, co ma na celu wyłapanie małych czynników pierwszych. Zamiast pisać oddzielną, kosztowną pętlę do wyliczania NWD przy pomocy algorytmu Euklidesa (która wprowadzałaby złożoność $O(r \log n)$), zintegrowałem to sprawdzenie bezpośrednio z pętlą poszukującą odpowiedniego parametru r w poprzednim kroku.

Ponieważ mój program testuje kolejne wartości sekwencyjnie (inkrementując r od wartości początkowej 2 wzwyż), jeśli liczba n posiada jakikolwiek różny od 1 wspólny dzielnik z a , to ten wspólny dzielnik wystąpi we wcześniejszej iteracji i z definicji NWD reszta z dzielenia n przez ten dzielnik będzie równa zero. Pozwoliło to na całkowite wyeliminowanie dedykowanej funkcji NWD ze struktury programu.

3.2. Pominiecie Redukcji Modulo w Pętli Mnożenia Wielomianów

Mnożenie wielomianów o dużych współczynnikach wiąże się z ogromnym obciążeniem programu i systemu operacyjnego, wynikającym z ciągłych operacji alokacji i zwalniania pamięci podręcznej dla struktur wielkich liczb biblioteki GMP. Aby zredukować ten narzut obliczeniowy, wykorzystałem skorzystałem z własności arytmetyki modularnej.

Zamiast wykonywać kosztowną redukcję modulo po każdym pojedynczym mnożeniu współczynników wewnątrz zagnieżdżonych pętli, pozwoliłem wewnętrznej strukturze na nieskrępowaną akumulację surowych sum. Maksymalna teoretyczna wartość takiego akumulatora wynosi $r \cdot n^2$, co bez problemu mieści się w pamięci struktur GMP bez ryzyka przepełnienia pamięci, ani zbyt wielkiego dodatkowego narzutu obliczeniowego. Dopiero po zsumowaniu wszystkich odpowiednich wyrazów dla danego stopnia wielomianu wykonuję pojedyncze wywołanie redukcji modulo. Rozwiązanie to zmniejszyło liczbę wywołań kosztownej funkcji `mpz_mod` z rzędu wielu milionów do zaledwie kilku tysięcy na jedną iterację.

3.3. Całkowitoliczbowa Metoda Babilońska

Obliczenie ostatecznego limitu pętli głównej wymaga wyciągnięcia pierwiastka kwadratowego z iloczynu funkcji φ Eulera oraz wyliczonej stałej. Użycie standardowej biblioteki zmiennoprzecinkowej `<math.h>` i rzutowania na typ `double` przy dużych danych wejściowych nieuchronnie skutkowałoby utratą precyzji bitowej lub błędami zaokrągleń.

Rozwiązałem ten problem, implementując całkowitoliczbową wersję metody pierwiastkowania Newtona-Raphsona. Zastosowałem zmodyfikowany warunek zakończenia pętli, dzięki temu algorytm nie wpada w pułapkę nieskończonej oscylacji między dwoma sąsiednimi stanami dyskretnymi, co jest powszechną wadą naiwnych implementacji tej metody na liczbach całkowitych.

4. Analiza Fragmentów Kodu Źródłowego

Poniżej przedstawiam kluczowy fragment implementacji operacji mnożenia dwóch wielomianów modulo. Zwracam uwagę na makro `mpz_sgn`, które wykonuje się w stałym czasie $O(1)$ poprzez bezpośredni odczyt wewnętrznego pola rozmiaru struktury GMP. Pozwala ono na natychmiastowe pominiecie zerowych elementów, co daje ogromne przyspieszenie obliczeń przy rzadkich wielomianach początkowych.

```

void poly_mod_multiply(
    poly_t result, const poly_t a,
    const poly_t b, const mpz_t modulus) {

    poly_memset(result, 0);

    for (size_t i = 0; i < a->length; i++) {
        // Optymalizacja O(1): natychmiastowe pominięcie pustych współczynników
        if(mpz_sgn(a->coefficients[i]) == 0) continue;

        for (size_t j = 0; j < b->length; j++) {
            if(mpz_sgn(b->coefficients[j]) == 0) continue;

            uint64_t new_degree = (i+j) % a->length;

            // Leniwe Modulo: bezpieczna akumulacja bez redukcji wewnątrz pętli
            mpz_addmul(result->coefficients[new_degree],
                a->coefficients[i],
                b->coefficients[j]);
        }

        // Ostateczna redukcja modulo przeprowadzana poza pętlą O(r^2)
        for(size_t i = 0; i < result->length; i++) {
            mpz_mod(result->coefficients[i], result->coefficients[i], modulus);
        }
    }
}

```

Drugim najważniejszym elementem systemu jest pętla wyszukująca właściwą wartość rzędu multiplikatywnego. To w niej zintegrowałem niejawne NWD:

```

uint64_t r = 2;
while(true) {
    if(mpz_cmp_ui(n, r) <= 0) return true;

    // Niejawne NWD, eliminujące klasyczny algorytm Euklidesa
    uint64_t remainder = mpz_fdiv_ui(n, r);
    if (remainder == 0) return false;

    bool is_order_large_enough = true;
    uint64_t current_remainder = remainder;

    // Rzutowanie na 128-bitów chroni przed przepełnieniem (overflow) przy potęgowaniu
    for (uint64_t k = 1; k <= bound; k++) {
        if (current_remainder == 1) {
            is_order_large_enough = false;
            break;
        }
        current_remainder =
            ((__uint128_t)current_remainder * (__uint128_t)remainder) % r;
    }
    if(is_order_large_enough) break;
    r++;
}

```

Zastosowanie natywnego typu `__uint128_t` chroni obliczenia potęgowe przed przepełnieniem bez konieczności kosztownego angażowania struktur GMP wewnątrz tej pętli. Stanowi to zarazem ostateczną barierę sprzętową kodu – wymusza, aby parametr r mieścił się w rejestrze 64-bitowym, co ustawia teoretyczny limit rozmiaru testowanych liczb na rząd 4 miliardów bitów.

5. Wydajność Obliczeniowa i Ograniczenia

Mimo że algorytm AKS udowodnił, iż problem testowania pierwszości leży w klasie P, asymptotyczna wielomianowość nie oznacza automatycznie natychmiastowego działania w praktyce inżynierskiej.

Zaimplementowana przeze mnie wersja wykorzystuje tak zwany „szkolny” algorytm mnożenia wielomianów o złożoności kwadratowej $O(r^2)$. Dla liczby pierwszej o rozmiarze 64 bitów, wartość parametru r oscyluje w granicach 5000. Podniesienie takiego wielomianu do kwadratu wymusza wykonanie minimum

25 milionów operacji dodawania i mnożenia w strukturach GMP na każdą iterację potęgowania binarnego. Z uwagi na konieczność wykonania około 96 takich kroków dla 64 bitów, procesor musi przetworzyć miliardy niskopoziomowych instrukcji.

W warunkach testowych, pełne sprawdzenie 32-bitowej liczby pierwszej zajmuje około 10 minut na procesorze AMD Ryzen 7 5800X. Pokazuje to ogromne wymagania obliczeniowe bezwarunkowego testu AKS i wyjaśnia, dlaczego w kryptografii produkcyjnej (np. generowanie kluczy RSA) wciąż powszechnie stosuje się probabilistyczny test Millera-Rabina. Wersje zaawansowane testu AKS wymagają wdrożenia mnożenia opartego na Szybkiej Transformacji Fouriera (FFT) w celu obniżenia złożoności operacji mnożenia wielomianów do $O(r \log r)$, co jednak znacząco komplikuje architekturę pamięciową.

6. Wpływ Hipotezy Agrawala na Efektywność Algorytmu

Głównym powodem tak potężnego narzutu obliczeniowego testu AKS jest konieczność wykonywania Kroku 5 dla ogromnej liczby niezależnych świadków a . Empirycznie można jednak łatwo dowieść, że niemal każda liczba złożona (nieposiadająca skrajnie małych czynników pierwszych) dyskwalifikowana jest już przy pierwszej próbie, czyli dla wartości $a = 1$.

Twórcy algorytmu sformułowali hipotezę (zwaną Hipotezą Agrawala), która mówi, że jeśli r jest liczbą pierwszą dzielącą n i spełniony jest warunek:

$$(X - 1)^n \equiv X^n - 1 \pmod{X^r - 1, n}$$

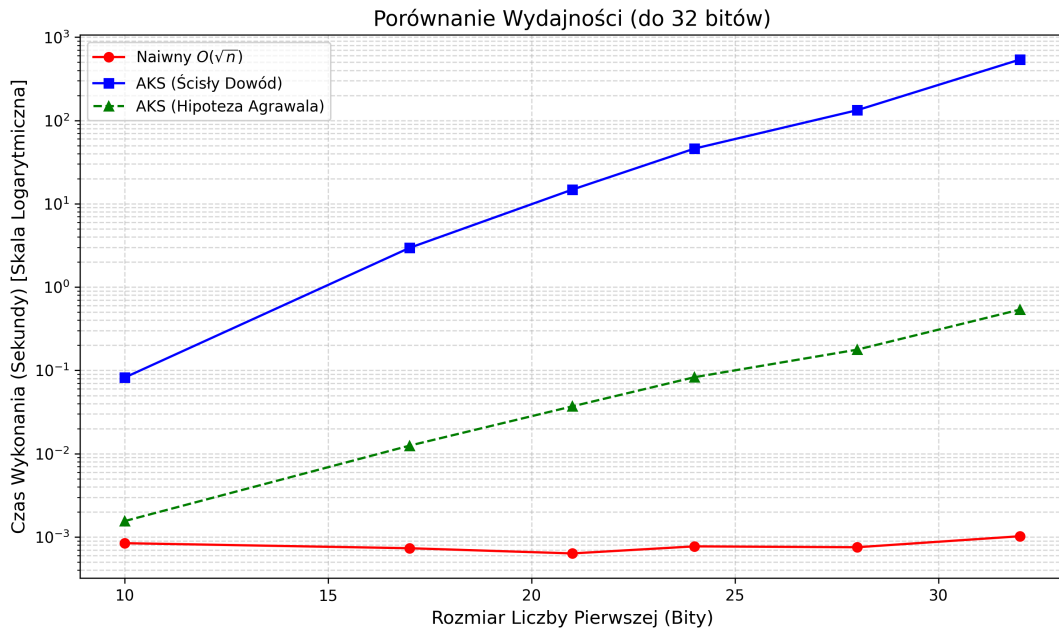
wtedy n jest liczbą pierwszą bądź zachodzi kongruencja $n^2 \equiv 1 \pmod{r}$. Założenie prawdziwości tej hipotezy redukuje poszukiwany limit iteracji a do wartości równej dokładnie jeden.

Zintegrowałem to podejście w mojej aplikacji, udostępniając użytkownikowi flagę `--use-unproven`. Eliminacja potężnej liczby pętli wymaganej do ścisłego, bezwarunkowego dowodu matematycznego skutkuje drastycznym skróceniem czasu procesorowego. Milionowa liczba pierwsza (ok. 24-bitowa) jest weryfikowana w czasie poniżej 80 milisekund, a testowanie liczb 128-bitowych staje się w pełni wykonalne na domowym komputerze osobistym.

7. Analiza Pomiarów Empirycznych

W celu dokładnego zbadania charakterystyki skalowania czasowego mojego programu, przeprowadziłem rozszerzoną serię pomiarów. Z uwagi na drastyczne różnice w złożoności obliczeniowej, wyniki wizualizacji podzieliłem na dwa niezależne wykresy (z osią czasu w skali logarytmicznej).

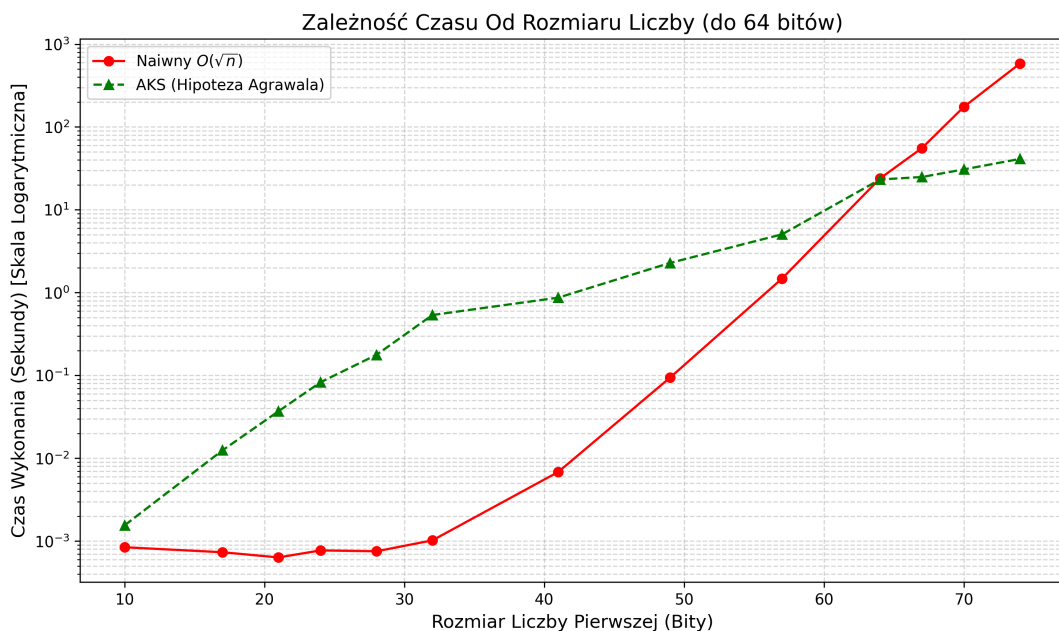
Pierwszy z nich analizuje liczby o rozmiarze do 32 bitów i zestawia ze sobą wszystkie trzy badane warianty.



Rysunek 1: Porównanie wydajności wszystkich trybów (do 32 bitów).

Jak widać na powyższym wykresie, klasyczny, bezwarunkowy algorytm AKS cechuje się stabilniejszą krzywą, jednak jego potężny narzut początkowy (związany z inicjalizacją struktur i ogromnymi limitami pętli) sprawia, że przegrywa on z podejściem naiwnym w badanym zakresie. Zastosowanie pełnego dowodu matematycznego dla liczb większych niż 32 bity staje się na domowym sprzęcie skrajnie niepraktyczne (wymagałoby godzin obliczeń), dlatego na tym etapie algorytm ten wyłączono z dalszych testów.

Drugi wykres przedstawia bezpośredni pojedynek między algorytmem naiwnym a testem AKS opartym na hipotezie Agrawala, rozszerzony aż do wielkich liczb 74-bitowych.



Rysunek 2: Zależność czasu od rozmiaru liczby i punkt przecięcia (do 74 bitów).

Rozszerzona analiza fenomenalnie obrazuje absolutną przewagę złożoności wielomianowej nad wykładniczą. Podejście naiwne wykazuje gwałtowny wzrost na wykresie. Przy 64 bitach obserwujemy przecięcie obu wykresów – oba algorytmy wymagały w okolicach 23 sekund na weryfikację liczby.

Jednak wystarczyło powiększyć badaną wartość do 74 bitów, by czas wykonania algorytmu naiwnego poszybował w dziesiątki minut z powodu konieczności wykonania miliardów iteracji. W tym samym czasie, czas rozwiązania dla zoptymalizowanego testu AKS wzrósł zaledwie marginalnie. Stanowi to definitywne i ostateczne zwycięstwo optymalizacji matematycznej nad surową siłą obliczeniową procesora.

8. Literatura

1. [Agrawal2004] M. Agrawal, N. Kayal, N. Saxena, „PRIMES is in P”, *Annals of Mathematics*, 160, 781-793, 2004.